Rates & Availability

Frontend implementation guide | Business usage, API flow and validation | 4 November 2026

Purpose. Change rates, meal/child configuration, sell status, booking restrictions and release timing without editing contract terms. Changes save **immediately**; this API has no draft/publish step. All 27 endpoints return HTTP 200 on success.

Pricing business rule

Daily Override > Season > Contract. Removing an override restores its fallback. Meal/child fields inherit independently. Zero is a valid price, not a removal.

Routes and permissions

Root R: /suppliers/ratesandavailability
Contract C: R/contracts/:contractId
Send Authorization: Bearer <supplier_token>; JSON bodies use Content-Type: application/json. Permission suffixes below follow **suppliers.rates_availability.**

Method + path	Purpose / permission suffix
GET R/hotels	Sellable hotel picker. **hotels.list**
GET R/contracts	All supplier contracts, newest first. **contracts.list**
GET R/contracts/:hotelId	Contracts for one hotel. **hotel_contracts.list**
GET C/rate-targets	Room/View targets and fixed meal offers. **rate_targets.list**
POST C/rates/context	Effective nightly room rates and supplements. **rates.context**
POST C/rate-overrides/preview	Validate/review proposed changes; no save. **rate_overrides.preview**
PUT C/rate-overrides	Atomically set/remove selected daily overrides. **rate_overrides.save**
GET C/rate-overrides	Paged existing overrides and current revision. **rate_overrides.list**

Guide map: sell controls on pages 7-9; restriction overrides 10-13; metadata/inventory reads 14; Manage Release 15-17; contract lifecycle and whole-contract stop sale 18. Each control family has its own revision.

New reads: POST C/supplements/context and GET C/supplement-overrides. Permissions: supplements.context / supplement_overrides.list. Full examples and reset behavior: pages 5-6.

ID distinction: GET R/contracts/10 means **hotelId=10**. GET R/contracts/42/rate-targets means **contractId=42**. Use variationId for edits, not roomId. GET R/:contractId reads contract detail (page 14).

Frontend sequence

1. Choose hotel/contract; load targets and check eligibility. **2.** Select rooms/dates and load context. **3.** Build operations and preview impact. **4.** Save the reviewed batch with expectedRevision. **5.** Refresh context/list and the control family revision.

Supplier admins need each endpoint permission; super admins retain the bypass. JWT scopes supplier ownership. Contract pickers include all statuses; check eligible and pricingIssues before enabling saves.

Targets and pricing context

Show what the selected rate actually changes before asking the supplier to save.

Target / operation	Meaning and frontend consequence
Base + Supplements **selected_room_total**	Target {variationId}. Entering 520 replaces this Room/View's room-only total. Do not add its room supplement again. Editing the base room alone does not change the other rooms.
Base + Supplements bulk **base_plus_room_supplement**	Apply an entered base to selected Room/View targets. The server adds each night's applicable room supplement and stores separate totals. Base 450 + seasonal supplement 70 = saved room amount 520.
Fixed Price offer **selected_room_total**	Target {variationId, mealPlan}. The entered rate replaces only this offer's room component; its meal charge remains additional. Other offers on the same Room/View are unchanged. Bulk base-plus-supplement basis is forbidden.

Meal scope: Base + Supplements must omit mealPlan. Fixed room-rate targets include an available mealPlan or room_only; daily-added offers are supported. Use the returned target combinations; do not invent offers. Targets include room names, view, kind, currency, contract dates, timezone, weekendDays, eligible, pricingIssues and ratesRevision.

Fixed target example: { "variationId": 18, "mealPlan": "bed_and_breakfast" }. Meal codes: room_only, bed_and_breakfast, half_board, full_board, iftar_and_suhoor; upgrades must be enabled for the selected nights. Prices use contract currency; do not send a currency override.

Request: POST C/rates/context

```
{
  "targets": [{ "variationId": 12 }],
  "startDate": "2027-06-01", "endDate": "2027-06-07",
  "weekdays": [0, 1, 2, 3, 4, 5, 6]
}
```

Alternatively send **dates: ["2027-06-01", "2027-06-05"]** and omit startDate/endDate. Both range ends are stay nights, inclusive. Keep dates as YYYY-MM-DD strings; avoid browser timezone conversions. weekdays uses 0=Sunday to 6=Saturday; omit it for all days. Duplicate explicit dates are deduplicated.

Read the response correctly

All responses use **{ status, message, data }**. Context data is **{ ratesRevision, eligible, nights[] }**. One representative night, shortened for clarity:

```
{
  "variationId": 12, "roomId": 11, "view": "haram",
  "mealPlan": null, "stayDate": "2027-06-01", "currency": "SAR",
  "roomAmount": 520, "source": "daily_override",
  "existingOverride": 520, "contractRoomAmount": 600,
  "fallbackAmount": 670, "fallbackSource": "season"
}
```

roomAmount/source: effective current room component and its origin.
fallbackAmount/fallbackSource: price after removing this override.
contractRoomAmount: original contract reference; **existingOverride=null** means none.
season, roomSupplement, mealPlans, selectedMeal, childPricing: relevant context with component sources.
roomSupplement.appliedSeparately=false: do not add it again.

Guest-dependent totals are not returned. Example: roomAmount 520 plus breakfast 45 per_person for two people gives 610 before child charges. Use selectedMeal for the chosen fixed offer; room_only has none. Do not sum all returned mealPlans together.

Preview, save and remove

Treat preview as the review screen and PUT as the immediate commercial change.

1. Preview an individual edit

```
POST C/rate-overrides/preview
{
  "operations": [{
    "action": "set", "targets": [{ "variationId": 12 }],
    "dates": ["2027-06-01", "2027-06-05"],
    "basis": "selected_room_total", "rate": { "amount": 520 }
  }]
}
```

Preview returns **ratesRevision**, **nights[]**, **affectedCount** (selected target/night cells), **replacedCount** (set cells already overridden), and **removedCount** (existing overrides selected for removal). For each night, show current roomAmount beside proposedAmount/proposedSource. Unticked nights are left unchanged.

2. Save the reviewed operations

PUT **C/rate-overrides** with **{ "expectedRevision": 0, "operations": [...] }**, replacing 0 with the revision returned by your latest preview and reusing the reviewed operations. Any invalid operation rejects the entire batch; there is no partial save.

Important: the save response retains the review's before-change roomAmount/source. Its **proposedAmount/proposedSource** represent the saved result, and ratesRevision is advanced. Refresh context/list after success; do not display the old roomAmount as the new price. Rates revisions are separate from contract expectedVersion.

3. Bulk base rates and weekday/weekend prices

```
{
  "operations": [{
    "action": "set",
    "targets": [{ "variationId": 12 }, { "variationId": 13 }],
    "startDate": "2027-06-01", "endDate": "2027-06-18",
    "basis": "base_plus_room_supplement",
    "rate": { "weekday": 450, "weekend": 550 }
  }]
}
```

Preview this body; add expectedRevision for PUT. Weekend days use targets.weekendDays: configured contract days, otherwise Thursday/Friday. **weekdays filters which nights change; rate.weekday/weekend chooses their amounts.** To apply one total to every selected room, use selected_room_total instead. For rate edits, "All rooms" means enumerate the returned targets; rate APIs have no allRooms flag. Sell controls use a separate selection on pages 7-9.

4. Remove overrides to restore the fallback

```
PUT C/rate-overrides
{
  "expectedRevision": 1,
  "operations": [{ "action": "remove",
    "targets": [{ "variationId": 12 }], "dates": ["2027-06-01"] }]
}
```

Omit rate and basis for remove. Missing overrides are harmless; even a successful no-op removal advances the revision. Fixed-offer removals must include mealPlan. There is no separate DELETE endpoint.

Validation and recovery

Validate locally for fast feedback, then use preview/save as the authority.

Rule	Required behavior
Contract / dates	Preview and save require a published active, scheduled or paused contract. Reject drafts, terminated/expired contracts and dates before today in the published hotel timezone. Every selected date must be within the contract term. Read context/list may inspect historical rates.
Date selection	Use explicit dates OR both range fields, never both. Ranges must be ordered and contain 1-366 nights before weekday filtering. Explicit lists contain 1-366 entries before deduplication. Empty weekday matches are invalid.
Targets / weekdays	1-10,000 target entries per selection; variationId is a positive integer up to 2147483647. Targets must belong to this contract. Supplement targets omit mealPlan for both models. Duplicate targets are rejected. weekdays contains 1-7 unique integer values in 0-6.
Operations / scale	1-100 operations per batch. Limits span both operation arrays: maximum 366 distinct dates and 10,000 expanded target/night cells. Operations must not overlap on the same variation + meal scope + date. Different offers are separate fixed scopes.
Rate format	Set requires basis + rate. Supply amount alone OR both weekday and weekend. Values must be finite JSON numbers from 0 to 10,000,000 with at most 2 decimals. Resulting bulk room totals must also fit these bounds. Remove forbids rate and basis.
JSON / revision	Unknown fields, numeric strings in bodies and invalid nulls are rejected. expectedRevision is a JSON integer from 0 to 2147483646. Never use a contract version here; obtain the current ratesRevision from context/preview.

Pickers and the existing-override list

GET R/hotels, R/contracts, R/contracts/:hotelId: only page/limit queries; filtering is client-side. Defaults: page 1, limit 20; maximum limit 100. Picker data has hotels[] or contracts[], meta and emptyState (no_hotels / no_contracts / null). Hotel names use nameEn/nameAr; contract rows include name, term, currency, model and status.

GET C/rate-overrides: page/limit plus optional startDate, endDate, variationId and fixed-offer mealPlan. Dates are inclusive; when both are supplied, startDate must not exceed endDate. This paged listing has no 366-date selection limit. Data contains overrides[], ratesRevision, meta and emptyState=no_overrides/null. Rows include amount, target/date, creator/editor and timestamps; base-model mealPlan is null. Refresh after saves.

Error handling

HTTP	Frontend action
400	Show validation feedback; fix selection, fields, dates or batch limits.
401 / 403	Restore supplier authentication or show missing endpoint permission.
404	Contract is missing or outside the supplier account; clear stale selection.
409	Stale revision, ineligible contract or pricing requiring review. Reload targets/context, re-preview and ask the supplier to review before saving again.

Network timeout on PUT: outcome may be unknown. Reload context/list before retrying. There is no Idempotency-Key contract here; a committed write advances the revision, so retrying its old revision conflicts rather than silently overwriting newer edits.

UI boundaries and acceptance

Check date/range filters, source fallback, replacements, offer isolation, bulk supplement arithmetic, removal and stale-save recovery. Clear targets when changing contracts; disable saves when eligible=false or pricingIssues remain.

Inventory editing, booking persistence and nationality pricing remain outside these APIs. Sell controls use pages 7-9; booking restriction overrides use pages 10-13. Do not invent booking counts or nationality context. Overrides preserve contract/season terms; future booking integration must retain confirmed price snapshots. Deploy the daily-rate migration before enabling these routes.

Daily meal and child configuration

Room-only, supplement-only and combined edits use the same atomic preview/save workflow.

Read date-specific configuration

POST C/supplements/context with targets [{variationId}] and the existing dates/range/weekday selection. Omit mealPlan for both models. The response has ratesRevision, eligible and nights[]. Each night includes meals (all four upgrades, even disabled ones), childPricing and availableTargets. Fixed daily-added meal offers appear here; GET rate-targets stays the original contract reference.

Each component returns effective values, **fieldSources**, **fallback** and nullable **dailyOverride** fields. A null daily field means inheritance, not zero. Context explains disabled availability and the value restored by a reset. For a complete commercial view, also call rates/context for the selected room/offer.

Preview or save supplement-only changes

Preview accepts the body below without expectedRevision. PUT C/rate-overrides accepts it with expectedRevision. Either operations, supplementOperations or both must be nonempty. Example introduces a meal and a complete child policy; real IDs and dates must pass eligibility checks.

```
{
  "expectedRevision": 0,
  "supplementOperations": [{
    "targets": [{ "variationId": 12 }],
    "startDate": "2027-06-01", "endDate": "2027-06-07",
    "mealChanges": [{ "type": "full_board", "action": "set",
      "enabled": true, "price": 150, "pricingBasis": "per_room" }],
    "childChanges": [
      { "category": "sharing_parents", "action": "set",
        "from": 0, "to": 5, "price": 0 },
      { "category": "shared_bed", "action": "set",
        "from": 6, "to": 11, "price": 50 },
      { "category": "extra_bed", "action": "set",
        "from": 6, "to": 11, "price": 120 }
    ]
  }]
}
```

Partial edits preserve omitted fields

Send only changed fields. A blank form field must be omitted, not sent as null or an empty string. Meal set may contain enabled, price and/or pricingBasis. Child set may contain from, to and/or price. Zero explicitly means free. For example, a price-only extra_bed edit keeps its daily/inherited age band. Changing breakfast to per_room keeps its daily/inherited price.

New meals: enable a supported upgrade with an explicit price and per_person/per_room basis when no contract values exist. enabled=false makes it unavailable only on selected Room/View nights. room_only cannot be configured or disabled.
Children: retain the three categories. Without a contract policy, introduce all three with complete age bands/prices. Child configuration is shared across every meal offer on the selected Room/View; occupancy limits do not change.

Resolution and preview results

Price fields: **Daily > Season > Contract**. Meal enabled/basis and child age bands: **Daily > Contract**. Base shared meals are fallback only; daily edits never spread to unselected Room/Views. New fixed offers inherit that variation's season/contract room component unless they have their own daily room rate.

Preview/save adds **supplementNights** with before values and a **proposed** configuration. **supplementCounts** reports affected Room/View nights, changed components, replaced rows and reset rows. Existing room counts stay unchanged. A combined request can enable a fixed meal and set its offer-specific room rate together; validation uses the final configuration.

Explicit reset and validation

Reset restores inheritance; it does not erase unrelated daily changes.

There is no separate reset endpoint

Use PUT C/rate-overrides. Existing operations/action=remove still removes **room pricing only**. A supplement reset uses action=reset plus the meal type or child category, with no value fields. Reset a meal to restore availability, basis and price fallback; reset a child category to restore all of its fields.

```
{
  "expectedRevision": 1,
  "supplementOperations": [{
    "targets": [{ "variationId": 12 }], "dates": ["2027-06-01"],
    "mealChanges": [{ "type": "full_board", "action": "reset" }],
    "childChanges": [
      { "category": "sharing_parents", "action": "reset" },
      { "category": "shared_bed", "action": "reset" },
      { "category": "extra_bed", "action": "reset" }
    ]
  }]
}
```

For **Undo all**, combine room removals with resets for every meal/category you intend to clear. Fixed room removals still need mealPlan; supplement resets never do. Missing rows are harmless, but every successful save advances ratesRevision. Disabling/resetting a meal retains its inactive room-rate row; re-enabling the meal reuses that rate.

Validation specific to meal and child changes

Rule	Required behavior
Supported fields	Meal upgrades: bed_and_breakfast, half_board, full_board, iftar_and_suhoor. Child categories: sharing_parents, shared_bed, extra_bed. No daily changes to room occupancy or supported code lists.
Value validation	enabled is a JSON boolean. pricingBasis is per_person or per_room. Prices use the existing 0-10,000,000 / two-decimal rule. Child ages are integers 0-17, from <= to, with to no higher than the selected room maximum child age (17 when unset).
Policy completeness	After merging all changes, the child policy must be complete in all three categories or absent. A partial daily-only policy is invalid. Reset all three together when no contract policy exists. Category age overlaps remain allowed.
Field preservation	Set must change at least one field; reset forbids value fields. Unknown fields, nulls and numeric strings are invalid. Omitted fields preserve earlier daily values. Enabled meals require an effective price and charging basis.
Overlap and limits	Room and supplement operations share the 100-operation / 366-date / 10,000-cell limits. Reject duplicate writes to the same meal type or child category on a Room/View night. Different components may be edited together.
Atomicity	One ratesRevision and one transaction cover every component. A stale revision returns 409. Invalid final configuration or a later write failure rolls back all room, meal and child changes.

Listing, availability and rollout

GET C/supplement-overrides supports page/limit, startDate/endDate, variationId and componentType=meal|child. It returns partial stored values with actor/timestamps, ratesRevision, meta and emptyState=no_supplement_overrides/null. A mealPlan filter is invalid. Use supplements/context for effective values.

Only available fixed offers may receive a new room-rate override. Existing inactive room-rate rows remain inspectable/removable. The new read routes need their separate permissions. Deploy migration **1793750400000-AddDailySupplementOverrides** after the existing daily-rate migration. No backfill is needed. Reset room rows for daily-added offers before reverting that migration.

Operational sell controls

Stop Sale, Open Sale and temporary On Request | One contract, one or many Room + View nights.

Choice	Effect on new bookings
Stop Sale	Blocks new bookings on selected nights/rooms. Keeps rates, inventory and existing confirmed bookings unchanged.
On Request	Requires supplier confirmation as an operational override. Keeps the Contract Booking Confirmation Type unchanged and preserves all other blockers.
Open Sale	Clears the selected operational override and restores normal contract behavior. A contract may still require confirmation or be blocked by inventory, release, cutoff, pauses or restrictions.

API map and permissions

Use the same **Contract C** prefix and supplier authentication. Permissions below follow **suppliers.rates_availability.** Every route returns the standard success envelope.

Method + suffix	Purpose / permission suffix
GET C/sell-control-targets	Configured variationId, roomId and view; eligible, timezone, bookingType, inherited SLA and sellControlsRevision. **sell_control_targets.list**
POST C/sell-controls/context	Read selected cells and current overrides; historical nights allowed. **sell_controls.context**
GET C/sell-controls	Paged live rules and exact remaining scope; optional date/variation filters. **sell_controls.list**
POST C/sell-controls/preview	Read-only ordered batch review, counts and before/after changes. **sell_controls.preview**
PUT C/sell-controls	Atomic Save, committed IDs and new sellControlsRevision. **sell_controls.save**

Single-night modal

1. Load context with variationIds:[12] and dates:["2027-06-01"]. Show the current override or normal contract behavior.
2. Choose a status; obtain rule SLA settings only when On Request has no contract SLA.
3. Apply sends one PUT operation with expectedRevision from context. Cancel performs no write.
4. Refresh context and live rules after success. Preserve form input and show feedback on failure.

Bulk modal and live-rule sidebar

Add stages an operation locally. Preview the entire ordered list; show changes[].before/after, counts (cells/nights/rooms), and replacedRuleIds. **Review & save all** sends the reviewed operations with the preview revision. Sidebar Edit/Delete are staged too; Cancel discards unsaved changes.

Boundary: single contract only. Hide the hotel-wide checkbox. Controls apply to every meal offer of the selected Room + View in either pricing model; never send mealPlan.

Sell-control request examples

Use variationIds or allRooms: true. These are separate APIs from rate/supplement edits.

Preview a range, then Save the reviewed batch

```
POST C/sell-controls/preview
{
  "operations": [
    { "action": "create", "status": "stop_sale", "allRooms": true,
      "startDate": "2027-06-01", "endDate": "2027-06-07" },
    { "action": "create", "status": "on_request", "variationIds": [12],
      "dates": ["2027-06-03"],
      "onRequestSettings": { "slaMinutes": 30,
        "expiryAction": "auto_reject" } }
  ]
}
```

The later On Request replaces Stop Sale only on Room/View 12 on 3 June. PUT **C/sell-controls** with the same operations plus **expectedRevision** from preview. Negative preview rule IDs are staged response identifiers; do not submit them as ruleId.

SLA inheritance and fallback

When the contract has SLA settings, On Request inherits them: **omit onRequestSettings** in the example above. Otherwise require both slaMinutes (integer 10-120) and expiryAction (auto_reject or escalate_to_hoteliana). Settings are forbidden on Stop Sale, Open Sale and remove. Responses report the effective settings and slaSource=contract|rule|null. Contract settings never change.

Open Sale clears only the selected cells

```
PUT C/sell-controls
{ "expectedRevision": 1, "operations": [
  { "action": "open_sale", "variationIds": [12],
    "dates": ["2027-06-03"] }
] }
```

Edit or remove an existing operational rule

```
{ "operations": [
  { "action": "edit", "ruleId": 45, "status": "stop_sale",
    "variationIds": [12, 13],
    "startDate": "2027-06-05", "endDate": "2027-06-07" }
] }
// To remove instead: { "action": "remove", "ruleId": 45 }
```

Edit submits the complete replacement scope and retains the live rule ID. Remove accepts only action/ruleId. Preview either body, then add expectedRevision for PUT. Removing clears only cells still owned by that rule; overwritten rules never revive.

Sell-control validation and recovery

Save is the live boundary. Keep sellControlsRevision separate from ratesRevision and contract version.

Rule	Frontend behavior
Scope / dates	variationIds must be unique configured IDs, or use allRooms:true exclusively. Dates OR an inclusive range; optional weekdays 0-6. All rooms expands again at Save. Reject empty matches and dates outside the term.
Eligibility / limits	Writes require published active/scheduled/paused contracts and nonpast nights in the published timezone. Maximum 100 operations, 366 distinct affected nights, 10,000 affected cells. Cleared edit/remove scope counts toward limits.
Overlap / clearing	Evaluate operations in request order; the latest choice replaces selected cells. Preserve unselected cells. Open Sale removes rather than storing an open status. Clear removed/overwritten cells permanently; do not reveal older overrides.
Independent revision	Send expectedRevision from sell-control context/preview. A stale revision returns 409. Refresh context/rules, re-preview the pending operations, and let the supplier review before resubmitting.
JSON / errors	Reject unknown fields, numeric strings, nulls and contradictory selections. 400: fix input/SLA/limits. 401/403: authentication/permission. 404: missing/foreign contract or no-longer-effective rule. 409: stale revision or ineligible contract.

Display existing live rules accurately

GET C/sell-controls accepts page/limit (1/20 defaults, limit max 100), startDate/endDate and variationId. Without date filters, rules must own a nonpast cell. Explicit dates enable historical inspection. The response contains rules[], meta, emptyState and sellControlsRevision. Each rule has audit fields, live, effective SLA, and its full remaining cells even when filters matched only part.

After partial replacement, display the exact remaining date segments and Room/Views. Edit/remove affect only nonpast owned cells. A spanning edit preserves historical status under a separate historical rule ID. Disable actions for historical-only rules. A null context override means normal contract behavior, not guaranteed sellability.

Immediate visibility, rollout and booking boundary

Save returns after commit and subsequent database reads see changes immediately; no background propagation or shared cache is involved. Refresh the visible grid after Save. If PUT times out, reload context/list before retrying because the write may have committed.

Deploy **1793836800000-AddOperationalSellControls** and grant the five new permissions before enabling these controls. The migration adds rules, effective cells and a dedicated revision without backfilling contract, rate or inventory values. Existing confirmed bookings are never rewritten.

Booking creation/confirmation and SLA execution are not implemented here. Future checkout must evaluate normal blockers, then call the exported SupplierSellControlsService.resolveStay in the same READ COMMITTED transaction, holding the parent contract lock through booking insertion. Any blocked night blocks the stay; otherwise any confirmation-required night makes the stay On Request. Nightly SLA policies are returned.

Acceptance checks

Verify single-night Apply; bulk Add/Cancel/review/save; all-room/weekday selection; overlap replacement; partial edit/remove; Open Sale with other blockers; contract/rule SLA; historical display; stale/concurrent saves; immediate refresh; supplier isolation; and unchanged pricing/inventory/confirmed bookings. Detailed API notes: documents/guides/operational-sell-controls-api.md.

Date-specific restriction overrides

Temporary minimum stay and check-in/check-out rules for Room + View dates, across every meal offer.

Independent fields and inherited rules

Suppliers can change **minimumStay**, **checkIn** and **checkOut** independently. Omitted fields and weekdays inherit. Minimum stay is an integer **0-365**: zero explicitly removes the requirement. **Open** explicitly overrides an inherited closure; it does not guarantee bookability.

Field precedence: Room-specific override > All-room override > Contract restriction > Default (minimum 0, check-in/check-out Open). Multiple selected Room/Views are room-specific. Shorter date ranges have no extra priority. Later operations replace overlapping fields only within the same layer.

Keep all-room fields underneath room-specific fields. Removing or expiring a specific field reveals the applicable all-room rule, then the contract rule. Superseded same-layer values never revive. This differs from the single operational layer used by sell controls.

API map and permissions

Use **C = /suppliers/ratesandavailability/contracts/:contractId**. Permission suffixes follow suppliers.rates_availability. Every response uses {status, message, data}.

Method + suffix	Purpose / permission suffix
GET C/restriction-targets	Room/View IDs, inheritedRules, weekendDays, timezone, eligible and restrictionsRevision. **restriction_targets.list**
POST C/restrictions/context	Selected cells with effective restrictions and fieldSources; historical dates allowed. **restrictions.context**
GET C/restriction-overrides	Paged rules with exact remaining field ownership and live/shadowed/history states. **restriction_overrides.list**
POST C/restriction-overrides/preview	Read-only batch review: changes, counts, replacedRuleIds and revision. **restriction_overrides.preview**
PUT C/restriction-overrides	Atomic immediate Save with expectedRevision; returns committed IDs and new revision. **restriction_overrides.save**

Single-night and bulk modal flow

1. Load targets; show inheritedRules as read-only contract rules and load context for the selection.
2. Pick dates and Room/Views; set only the desired fields. Show weekendDays from the contract.
3. **Add** stages locally. Stage sidebar Edit/Delete too. Cancel performs no write.
4. Preview the ordered batch and show its before/after fields, sources and impact.
5. Review & save all sends the same operations with the preview revision; refresh the grid and list.

For single-night Apply, use one operation and the current context revision. Targets apply to this contract only; never send mealPlan or a hotel-wide selection. Contract restrictions are edited through the existing draft-contract flow, not these override routes.

Restriction selection and context

Keep minimum-stay weekend scope separate from arrival/departure weekday changes.

Choose dates and Room + View targets

```
POST C/restrictions/context
{
  "variationIds": [12, 13],
  "startDate": "2027-06-01", "endDate": "2027-06-07"
}
```

Use **variationIds** OR **allRooms:true**, exclusively. All rooms expands to configured variations again at Save; it does not include other contracts or future variations. Use **dates:["2027-06-01"]** OR both range fields. Both ends are inclusive dates. Optional weekdays filters the selection; 0=Sunday through 6=Saturday.

Minimum stay versus weekday closures

Input	Meaning
minimumStay	{minNights:0..365, appliesOn:every_day|weekend_days}. Weekend-only uses the contract weekendDays, falling back to Thursday/Friday. Nonmatching dates inherit minimum stay.
checkIn / checkOut	Arrays of {weekdays:[0..6], status:open|closed}. Groups within each field must be disjoint; use separate groups for different states. Omitted weekdays inherit. Weekend-only minimum stay does not filter these changes.
Blank / zero / Open	Omit unchanged fields. Do not submit null or empty arrays. Zero and Open are explicit values, not reset actions. At least one supplied field must match a selected date.

Render effective values and their sources

Context data includes contractId, timezone, eligible, weekendDays, inheritedRules, restrictionsRevision and cells[]. One cell, shortened for clarity:

```
{
  "variationId": 12, "stayDate": "2027-06-01",
  "restrictions": {
    "minNights": 0, "checkIn": "open", "checkOut": "closed",
    "fieldSources": {
      "minNights": {"source":"room_override", "ruleIds":[45]},
      "checkIn": {"source":"all_rooms_override", "ruleIds":[42]},
      "checkOut": {"source":"contract", "ruleIds":[8]}
    }
  }
}
```

Source values are **room_override**, **all_rooms_override**, **contract** or **default**. A default has an empty ruleIds array. Contract rule IDs and operational rule IDs belong to different sources: label them with their source.

Display inheritedRules with their dates, roomId (null means all rooms), minNights, appliesOn, closed weekday arrays and isActive. Resolution ignores inactive/out-of-scope rules. Matching room-specific contract rules take priority per field; within that tier use the highest minimum and combine closed weekdays. Contract appliesOn filters minimum stay only.

Restriction preview, save and edit

Keep restrictionsRevision separate from ratesRevision, sellControlsRevision and the contract version.

Stage a range and a more specific exception

```
POST C/restriction-overrides/preview
{
  "operations": [
    { "action":"create", "allRooms":true,
      "startDate":"2027-06-01", "endDate":"2027-06-07",
      "minimumStay":{"minNights":3,"appliesOn":"weekend_days"},
      "checkIn":[{"weekdays":[4,5],"status":"closed"}] },
    { "action":"create", "variationIds":[12],
      "dates":["2027-06-03"],
      "minimumStay":{"minNights":0,"appliesOn":"every_day"},
      "checkIn":[{"weekdays":[4],"status":"open"}] }
  ]
}
```

Room/View 12 on Thursday 3 June has minimum 0 and check-in Open, even if a later all-room operation is submitted. Other dates/fields follow their applicable layer. Preview reports changes[].before/after (including fieldSources), counts.cells/nights/rooms, replacedRuleIds and restrictionsRevision. Touched shadowed/no-op cells can be counted even when effective values do not change.

Save: PUT C/restriction-overrides with the same operations plus expectedRevision from preview. It returns the new restrictionsRevision, committed positive operation IDs and effective after values. Negative preview IDs are response-only; never send them as ruleId. Refresh context/list after success.

Edit or remove a live rule

```
{ "operations": [
  { "action":"edit", "ruleId":45, "variationIds":[12],
    "dates":["2027-06-05"],
    "minimumStay":{"minNights":2,"appliesOn":"every_day"} }
] }
// Remove instead: { "action":"remove", "ruleId":45 }
```

Preview either operation, then add expectedRevision for PUT. Edit supplies the complete replacement scope and desired definition; omitted fields inherit. It clears the old nonpast owned fields before applying the replacement and keeps the live ID. Scope may change between explicit rooms and all rooms.

Remove takes **only action and ruleId**. It clears remaining nonpast owned fields, leaving other rules intact. Specific-field removal reveals a lower layer. Historical-only, fully replaced, removed or foreign-contract rules cannot be edited/removed.

Restriction recovery and acceptance

Expiration follows date scope automatically. Existing confirmed bookings are never rewritten.

Existing-rule sidebar and history

GET C/restriction-overrides accepts page/limit (1/20 defaults, limit max 100), optional startDate/endDate and variationId. Without dates it returns rules owning nonpast fields, including shadowed rules. Date filters enable historical inspection. The response has rules[], inheritedRules, meta, emptyState=no_restriction_overrides|null and restrictionsRevision.

Each rule returns its **full remaining cells**, not just the filter match: field, value, layer, ruleId and status. value is minimum nights or 0=Open/1=Closed. Status is effective, shadowed or historical. live means at least one nonpast owned field. Display exact disconnected segments; disable actions for historical-only rules. A spanning edit preserves past values under a separate historical rule ID.

Validation and recovery

Rule	Frontend behavior
Eligibility / dates	Writes and preview require a published active, scheduled or paused contract. Use nonpast dates inside its term, based on the published hotel timezone. Reads may inspect past dates. Paused contracts remain paused.
Scale / fields	1-100 operations, 366 distinct affected dates and 10,000 affected Room/View-date cells. Include scope cleared by edits/removals. Fields/layers do not multiply the room-night count. Reject empty matches, duplicate weekday groups, nulls, unknown fields and numeric strings.
409 / timeout	Refresh context/list, re-preview and review again before saving a stale revision. Never retry silently with a new revision. A timeout may follow a committed write; reload before retrying. No Idempotency-Key is accepted.
400 / 401 / 403 / 404	400: fix fields, scope or limits. 401/403: restore authentication or endpoint permission. 404: missing/foreign contract, missing live rule or historical-only rule; reload the sidebar.

Meaning for new booking decisions

The stay must meet the **highest effective minimum across every occupied night**. Evaluate check-in on arrival and check-out on departure, not on intermediate nights. Departure is not an occupied night. A departure after the final contract night is unrestricted by this contract. Explicit Open does not clear other booking blockers.

Future booking consumers evaluate each Room/View stay with SupplierRestrictionOverridesService.resolveStay and merge blockers (minimum_stay, check_in_closed, check_out_closed) into normal decisions before sell-control resolution. Use the same READ COMMITTED transaction and parent-contract lock through booking insertion. Booking persistence is not implemented here; retain confirmed snapshots.

Rollout and acceptance

Deploy **1793923200000-AddOperationalRestrictionOverrides** with schema synchronization disabled and grant all five permissions before enabling the modal. The additive migration has no contract/rate/inventory backfill. Effective reads change after Save commits; no expiration job or cache propagation is needed.

Verify single/range/multiple/all-room selection; zero and Open; independent weekday groups; configured/fallback weekends; room priority even after later all-room edits; same-layer replacement; removal/expiry fallback; historical preservation; stale and concurrent saves; supplier isolation; and unchanged contract/rate/inventory/confirmed data. Full reference: documents/guides/restriction-overrides-api.md.

Metadata and contract inventory reads

Latest read endpoints alongside the calendar pickers. These return stored configuration and stock, not booking decisions.

Method + path	Purpose / permission suffix
GET R/metadata	Countries of this supplier's sellable hotels. **metadata.view**
GET R/:contractId	Full contract detail plus hotel identity and stored nightly inventory. **contracts.view**

Country metadata for filters

GET R/metadata returns data.countries[] with code, nameEn and nameAr. Countries are distinct, sorted by English name, and limited to the supplier's sellable linked hotels. Hotels without a country are skipped; no matches returns an empty array. This endpoint is not paginated.

Load one contract with its stored nightly stock

GET R/:contractId returns the same full detail as GET /suppliers/hotel-contracts/:id, plus hotel {id, nameEn, nameAr} and inventory. Read from a single snapshot. There are no query parameters; another supplier's contract returns 404.

```
{
  "id": 42,
  "hotel": { "id": 10, "nameEn": "Hotel", "nameAr": null },
  "publicationStatus": "ready",
  "inventory": {
    "pools": [{
      "id": 1, "poolKey": "shared_pool", "mode": "shared_pool",
      "variationId": null,
      "nights": [{"stayDate":"2027-06-01",
                  "capacity":50,"overbookingLimit":2}]
    }],
    "caps": [{ "variationId":12, "poolId":1,
      "nights":[{"stayDate":"2027-06-01","maxRooms":12}] }]
  }
}
```

inventory.pools describes the finalized pool mode and its stored nightly capacity/overbooking limit. capacity is null for free sale. inventory.caps contains per-variation nightly caps within a shared pool. Both arrays are empty before finalization (publicationStatus=not_finalized). These reads do not edit inventory or calculate remaining bookable rooms.

Keep IDs and effective controls separate

GET R/contracts/:hotelId lists a hotel's contracts. **GET R/:contractId** reads one contract. **C = R/contracts/:contractId** is the prefix for operational contexts and override writes. A variationId identifies Room + View, not hotelId, contractId or the room-library roomId.

Contract detail shows inherited configuration. For the effective calendar overlay, load rate/supplement context, sell-control, restriction and release context for the visible selection. Keep ratesRevision, sellControlsRevision, restrictionsRevision and releaseRevision independent. Do not treat stored capacity or an Open restriction as a guarantee of sellability.

Manage Release: scope and priority

Temporary release timing for selected stay nights and Room + View variations, across every meal offer.

Use this modal to change when unsold rooms return to the hotel, without editing the published contract. The override changes **timing only**. After-release Stop Sale or On Request, and its SLA settings, always come from the contract. Existing confirmed bookings remain unchanged.

Routes and permissions

Method + path	Purpose / permission suffix
GET C/release-targets	Targets, inheritedPolicy, SLA, timezone, eligible, canOverride and releaseRevision. **release_targets.list**
POST C/release/context	Selected cells with effective timing, source, ruleId and local deadline. **release.context**
GET C/release-overrides	Paged rules and full remaining ownership, including shadowed/history. **release_overrides.list**
POST C/release-overrides/preview	Read-only review: before/after timing, counts and replacement IDs. **release_overrides.preview**
PUT C/release-overrides	Atomic immediate Save using expectedRevision; returns new releaseRevision. **release_overrides.save**

Choose nights and rooms

Select **dates:["2027-06-01"]** for one night, multiple explicit dates, or inclusive **startDate/endDate**. Never combine the two date forms. Optional weekdays uses unique integers 0=Sunday through 6=Saturday and filters the selected nights.

Choose **variationIds:[12]** for one Room + View, **variationIds:[12,13]** for multiple, or **allRooms:true**. Never send both room selectors. All rooms expands the configured contract variations at preview/save. Do not send mealPlan, library roomId or hotel-wide scope.

Timing and inherited behavior

number_of_days: daysBeforeCheckIn is a required integer **1-365**, superseding the 1-30 hint in the mockup. **same_day:** omit daysBeforeCheckIn entirely; null and zero are invalid. Both require strict local releaseTime in HH:MM, 00:00-23:59. afterReleaseAction and onRequestSettings are forbidden override inputs.

Priority: Room-specific override > All-room override > Contract. Explicit multiple-room selection uses the room-specific layer. A later all-room save cannot hide a room-specific value. Within a layer, later operations permanently replace overlap; a shorter date range adds no priority.

Outside the selected period, a rule has no effect. Removing/expiring room timing reveals applicable all-room timing, then the contract. Replaced same-layer rules never revive. Booking cutoff remains independent. No expiration cleanup job is needed.

Eligibility

Create/edit need a configured contract release policy, publication, captured publishedTimezone and active/scheduled/paused status. canOverride combines publication eligibility with inherited policy availability. Writes use today-or-later dates inside the term; historical context/list reads are allowed. Add/Edit/Delete stage locally; Cancel discards the list.

Release context, preview and save

Review the full pending list before saving. releaseRevision belongs only to Manage Release.

1. Read effective timing

```
POST C/release/context
{ "variationIds": [12], "dates": ["2027-06-01"] }
```

Each data.cells[] row carries contractId, variationId, stayDate and release. Metadata includes inheritedPolicy, onRequestSettings, timezone, weekendDays, eligible, canOverride and releaseRevision. A shortened effective release value:

```
{
  "type": "number_of_days", "daysBeforeCheckIn": 3,
  "releaseTime": "18:00", "source": "room_override",
  "ruleId": 45, "afterReleaseAction": "on_request",
  "onRequestSettings": {
    "slaMinutes": 30, "expiryAction": "auto_reject"
  },
  "deadline": {
    "date": "2027-05-29", "time": "18:00",
    "timezone": "Asia/Riyadh"
  }
}
```

source is room_override, all_rooms_override, contract or default. Contract/default ruleId is null; without a policy, timing/deadline are null. Deadline subtracts calendar days from **each stay date**, then applies releaseTime in publishedTimezone. It is a local date/time plus IANA timezone, not a UTC instant or proof of released stock. Future execution must define DST gap/fold handling.

2. Preview the ordered pending list

```
POST C/release-overrides/preview
{ "operations": [
  { "action": "create", "allRooms": true,
    "startDate": "2027-06-01", "endDate": "2027-06-07",
    "type": "number_of_days", "daysBeforeCheckIn": 3,
    "releaseTime": "18:00" },
  { "action": "create", "variationIds": [12, 13],
    "dates": ["2027-06-03"], "type": "same_day",
    "releaseTime": "14:00" }
] }
```

Preview returns operations[], changes[].before/after with sources/deadlines, counts.cells/nights/rooms, replacedRuleIds and releaseRevision. Room/Views 12 and 13 use same-day timing on 3 June; remaining selected scope uses the all-room rule. Counts include touched shadowed/no-op cells. Negative staged rule IDs are response-only.

3. Review and save all

PUT **C/release-overrides** with the same operations plus **expectedRevision** from release context/preview. Save commits immediately and atomically, remaps new IDs to positive values and advances releaseRevision. Refresh context/list. Single-night Apply can submit one operation with its current revision. Do not use contract version or the revision of another control family.

Release editing, recovery and rollout

Preserve scope, ownership and history while keeping contract rules visible as inherited.

Sidebar, edit and delete

GET C/release-overrides accepts page/limit (defaults 1/20, limit max 100), optional startDate/endDate and variationId. Default listing finds rules owning nonpast cells; explicit date filters permit history. A matching rule returns its **full remaining scope**, including disconnected segments outside the filter slice. Response includes rules[], meta, emptyState=no_release_overrides|null and releaseRevision.

Show inheritedPolicy separately as read-only **From the contract**. Rule rows include timing, layer, audits, live and cells. Cell status is effective, shadowed or historical. live means at least one nonpast owned cell; a shadowed all-room rule can still be edited. Disable Edit/Delete for historical-only rules.

```
{ "operations": [{ "action": "edit", "ruleId": 45,
    "variationIds": [12], "dates": ["2027-06-05"],
    "type": "same_day", "releaseTime": "15:00" }] }

{ "operations": [{ "action": "remove", "ruleId": 45 }] }
```

Edit supplies the complete replacement selection and timing. It clears old nonpast ownership before applying the new scope and retains the live ID. Past timing/ownership is archived under a separate ID when needed. Remove takes **only action/ruleId** and preserves past ownership. Both are batch operations: preview first, then add expectedRevision for PUT. There is no DELETE route.

Validation and recovery

Check	Required frontend behavior
Limits	1-100 operations; at most 366 distinct affected nights and 10,000 expanded variation/date cells. Include old nonpast scope cleared by edits/removals. Date ranges/arrays allow at most 366 nights before filtering.
400 / 401 / 403	400: invalid/contradictory timing, unknown fields, numeric strings, dates, scope or limits. 401: authenticate. 403: supplier role or endpoint permission denied.
404 / 409	404: missing/foreign contract or noneditable rule (removed, fully replaced or historical-only). 409: ineligible contract, missing inherited policy for create/edit, or stale releaseRevision.
Stale or timeout	On 409, reload, re-preview the intended list and show updated impact. Never silently bump revision/retry. No Idempotency-Key; after a timeout reload before deciding whether to retry.

Docker deployment and business boundary

Development schemaMaintenance defaults to true. After verification and backup, the deployment runs **node dist/database/rates-availability-deployment.js --migrate** inside the **release Docker image**, with **DB_SYNC=false**, before starting the API. The scoped runner includes **1794096000000-AddOperationalReleaseOverrides** after the four earlier operational migrations. Its preflight checks the new tables and scope trigger; do not use schema synchronization.

Grant the five release permissions through supplier profiles before enabling the UI. This delivers rule management and a manager-aware effective resolver, with no stock-release worker or checkout integration. Future booking consumers must resolve release alongside cutoff, inventory, restrictions and sell controls under the shared parent lock. Confirmed booking snapshots remain unchanged.

Acceptance: single/range/weekday and one/multiple/all-room selection; 1-365 days and Same day; room priority; permanent same-layer replacement; lower-layer/contract fallback; exact remaining scopes/history; stale/concurrent saves and rollback; timezone-local deadlines; unchanged contract, inventory and confirmed data. Full reference: documents/guides/release-overrides-api.md.

Contract lifecycle and whole-contract stop sale

Contract menu actions preserve published terms, stock and confirmed bookings.

Pause, Resume and Terminate

Pause: published Active/Scheduled contracts whose term has not ended. Hidden from future agent discovery and blocked for new sales.
Resume: Paused only. Resolve Scheduled before the term, Active during it, or Expired after it, using the captured publication timezone. Recheck publication and supplier/hotel/link eligibility before reopening.
Terminate: published Active/Scheduled/Paused only; irreversible. Existing confirmed bookings are honoured.

Prefix H: /suppliers/hotel-contracts/:id. All operations below return HTTP 200 in {status,message,data}. Use supplier JWT and application/json. No query parameters.

POST suffix	Body / permission suffix
H/pause	{expectedVersion, reason?} **pause**
H/resume	{expectedVersion, reason?} **resume**
H/terminate	{expectedVersion, reason?} **terminate**
H/stop-sale/preview	{startDate, endDate, reason?} **stop_sale.preview**
H/stop-sale	{startDate, endDate, expectedRevision, reason?} **stop_sale.save**

Permission prefix: **suppliers.hotel_contracts.** Grant the new keys to admin profiles; Super Admin retains the normal bypass. Lifecycle responses return full contract detail and increment version once. Optional reasons are trimmed, nonblank when supplied, maximum 1000 characters; null is rejected. Actor/time/reason are audited.

Whole-contract stop sale: preview, then save

Choose inclusive nonpast dates inside a published Active/Scheduled/Paused contract. Preview expands every configured Room + View across all meal offers and returns **nightCount, variationCount, cellCount, replacedControlCounts** (stopSale/onRequest), **sellControlsRevision**, dates, timezone and reason. Save the same selection with **expectedRevision**; its compact response adds **ruleId**.

This action keeps lifecycle status and contract version unchanged. Only sellControlsRevision increments. Atomic bulk saves support more than 366 nights / 10,000 cells; existing interactive limits stay unchanged. Later selected-room/date **Open Sale** can reopen cells, without restoring superseded rules. Resume does not clear existing overrides.

Recovery, deployment and booking integration

400/415: correct fields/dates or JSON content type. **401/403:** authentication/permission. **404:** missing/foreign contract. **409:** stale version/revision or invalid transition; reload and deliberately reapply. No automatic retry using a newer revision.

Docker verifies prerequisites and automatically installs the two additive reason migrations before API startup with **DB_SYNC=false**; failure blocks startup. Legacy baseline repair still uses explicit maintenance. Keep the parent lock through future booking insertion; the shared stay resolver retains lifecycle and all upstream blockers. No agent discovery or booking persistence endpoint is added.